

1 Recap

- A language L is in $\text{DTIME}(f(n))$ iff there is a Turing machine (TM) V that runs in time $O(f(n))$ such that

$$x \in L \iff V(x) = 1.$$

- $P = \bigcup_{c \geq 1} \text{DTIME}(n^c)$.
- A language L is in $\text{NTIME}(f(n))$ iff there is a polynomial-time TM V such that

$$x \in L \iff \exists w, |w| \leq f(|x|), V(x, w) = 1.$$

- $\text{NP} = \bigcup_{c \geq 1} \text{NTIME}(n^c)$.
- A language L is **NP-complete** if $L \in \text{NP}$ and every $L' \in \text{NP}$ is reducible to L in polynomial time, written $L' \leq_p L$.

2 The Time Hierarchy Theorem

Definition 1. A function $f : \mathbb{N} \rightarrow \mathbb{N}$ is **time-constructible** if $f(n) \geq n$ and there is a TM that, on input 1^n , outputs the binary representation of $f(n)$ in time $O(f(n))$.

Theorem 2. *There is a universal Turing machine U that can simulate any other Turing machine given its bit representation. Given a pair of bit strings (x, α) as input, U simulates $M_\alpha(x)$ within running time of $O(T(|x|) \log T(|x|))$ if the running time of M_α was $T(|x|)$.*

Theorem 3 (Time Hierarchy Theorem [AB09, Theorem 3.1]). *Let $f, g : \mathbb{N} \rightarrow \mathbb{N}$ be time-constructible functions with $f(n) \log f(n) = o(g(n))$. Then*

$$\text{DTIME}(f(n)) \subsetneq \text{DTIME}(g(n)).$$

Proof sketch of Theorem 3. We build a TM D that, on input x of length n :

1. Uses time-constructibility of g to compute $g(n)$ and sets a clock of length $c \cdot g(n)$, for a constant c fixed below.
2. Treats x itself as (an encoding of) a TM M_x , and run the universal TM U to simulate M_x on input x .
3. If the simulation halts within the clock $c \cdot g(n)$ and outputs a bit b , D outputs $1 - b$; if the clock runs out first, D outputs 0.

Let $L(D) := \{x \mid D(x) = 1\}$ be the language decided by D .

(i) $L(D) \in \text{DTIME}(g(n))$. By the construction of $L(D)$ and the time-constructibility of g , D is the TM that runs in time $O(g(|x|))$ such that $x \in L(D) \iff D(x) = 1$.

(ii) $L(D) \notin \text{DTIME}(f(n))$. Suppose toward a contradiction that some TM M decides $L(D)$ in time $O(f(n))$. Let x be the binary representation of M , denoted by $\langle M \rangle$, and $n := |x|$. Simulating M on input x through the universal machine costs $O(f(n) \log f(n))$ steps, which by hypothesis is $o(g(n))$ and hence eventually smaller than D 's clock $c \cdot g(|x|)$. So for all large enough n , D on input x finishes the simulation $M(x)$ and outputs $1 - M(x)$. But M and D decide the same language $L(D)$, so $M(x) = D(x)$ for every x , in particular for $x = \langle M \rangle$. This gives $M(x) = 1 - M(x)$, a contradiction. $\square$

3 NP-Completeness: 3-SAT and the Cook–Levin Theorem

Definition 4. Fix Boolean variables $x_1, \dots, x_m$. A **literal** v_{i_j} is a variable x_{i_j} or its negation $\bar{x}_{i_j}$. A **3-clause** is a disjunction of exactly three literals,

$$(v_{i_1} \vee v_{i_2} \vee v_{i_3}), \quad i_1, i_2, i_3 \in [m].$$

A **3-CNF formula** Φ is a conjunction of 3-clauses, $\Phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$. An **assignment** $\sigma : \{x_1, \dots, x_m\} \rightarrow \{0, 1\}$ gives Φ a value $\Phi(\sigma) \in \{0, 1\}$ in the natural way.

Definition 5.

$$3SAT = \{ \langle \Phi \rangle : \Phi \text{ is a 3-CNF formula and } \exists \sigma, \Phi(\sigma) = 1 \}.$$

Theorem 6 (Cook–Levin [AB09, Theorem 2.10]). *3SAT is NP-complete.*

Proof sketch of Theorem 6. Membership: $3SAT \in \text{NP}$. Given Φ , if $\Phi \in 3SAT$, then there exists a witness $|\sigma| = O(|\Phi|)$ such that $\Phi(\sigma) = 1$ and checking that $\Phi(\sigma) = 1$ takes time linear in $|\Phi|$.

Hardness: every $F \in \text{NP}$ satisfies $F \leq_p 3SAT$. Since $F \in \text{NP}$, fix a polynomial p and a poly-time verifier V with

$$x \in F \iff \exists w, |w| \leq p(|x|), V(x, w) = 1.$$

The reduction R , on input x , does the following.

1. **TM** $\rightarrow$ **circuit**. Build a Boolean circuit $C_x(\cdot) := V(x, \cdot)$ that, on an input w of length $p(|x|)$, outputs the same bit as $V(x, w)$. This is possible because any poly-time computation can be simulated by a Boolean circuit whose size is polynomial in the running time; here $|C_x| = \text{poly}(|x|)$.
2. **Circuit** $\rightarrow$ **3-CNF**. Introduce one Boolean variable per wire of C_x (the $p(|x|)$ input wires become the free variables of w ; every other wire gets a fresh auxiliary variable). Each gate has fan-in at most 2 (AND, OR, or NOT); for each gate, add the handful of 3-clauses that force its output-wire variable to equal the correct Boolean function of its input wire(s). This amounts to excluding the rows of the gate's truth table (at most $2^3 = 8$ rows) that violate the gate's behavior. Finally, add clauses forcing the designated output wire to be 1.
3. **Output**. Let Φ_x be the conjunction of all these clauses. Since $|C_x| = \text{poly}(|x|)$ and each gate contributes $O(1)$ clauses, $|\Phi_x| = O(|C_x|) = \text{poly}(|x|)$, so $R(x) := \langle \Phi_x \rangle$ is computable in polynomial time.

By construction, Φ_x is satisfiable iff C_x has a satisfying assignment to its input wires w , iff $\exists w, V(x, w) = 1$, iff $x \in F$. Hence $F(x) = 3SAT(R(x))$ for all x . $\square$

4 P versus NP: Solving versus Verifying

The P vs. NP question can be phrased as “solving vs. verifying”: languages in NP are exactly those for which a proposed witness (a solution, a certificate, a proof) can be *checked* efficiently, whereas P asks that a solution can be *found* efficiently from scratch. Checking a proof is intuitively much easier than discovering one, which is the main reason many people believe $P \neq NP$; yet after decades of effort the question remains open.

Remark 7. If $P = NP$, the consequences would be dramatic. In particular, mathematical theorem-proving would (in a precise, though idealized, sense) become automatable: any theorem whose proof is not “unreasonably” long relative to its statement could, in principle, be found in time polynomial in the statement’s length, since verifying a candidate proof is itself a polynomial-time task.

Remark 8 (Nondeterministic machines and verifiers). A nondeterministic TM (NTM) may have several possible transitions out of a given configuration; its computation branches into a tree of possibilities, and the machine accepts an input if *some* branch of the tree accepts. The verifier-based definition of NP above is equivalent to this machine-based one: the sequence of nondeterministic choices along an accepting branch plays exactly the role of the witness w , and following that branch step by step is exactly what the verifier $V(x, w)$ does.

5 EXP and NEXP

Definition 9.

$$\text{EXP} = \bigcup_{c \geq 1} \text{DTIME}(2^{n^c}), \quad \text{NEXP} = \bigcup_{c \geq 1} \text{NTIME}(2^{n^c}).$$

Equivalently, in verifier language: $L \in \text{NEXP}$ iff there is a constant c and a verifier V (running in time polynomial in the combined length of its two inputs) such that

$$x \in L \iff \exists w, |w| \leq 2^{|x|^c}, V(x, w) = 1.$$

Because the witness w may be as long as $2^{|x|^c}$, running V on (x, w) can itself take time exponential in $|x|$, even though V ’s running time is only *polynomial in the length of its own input*.

We have the containments

$$P \subseteq NP \subseteq \text{EXP} \subseteq \text{NEXP},$$

where $NP \subseteq \text{EXP}$ holds because a brute-force algorithm can try all $2^{p(|x|)}$ ($p(\cdot)$ is a polynomial) candidate witnesses of an NP verifier in exponential time.

Remark 10. By the Time Hierarchy Theorem, $P \subsetneq \text{EXP}$ strictly. Consequently, if it turned out that $NP = \text{EXP}$, that alone would already imply $P \neq NP$. Whether $NP = \text{EXP}$ is itself a wide open question, just like P vs. NP.

6 P = NP Implies EXP = NEXP

Theorem 11 ([AB09, Theorem 2.22]). *If $P = NP$, then $\text{EXP} = \text{NEXP}$.*

Proof. Since $\text{EXP} \subseteq \text{NEXP}$ always holds, it suffices to show $\text{NEXP} \subseteq \text{EXP}$. Let $L \in \text{NEXP}$ be arbitrary, witnessed by a constant c and a verifier V with

$$x \in L \iff \exists w, |w| = 2^{|x|^c}, V(x, w) = 1$$

(padding the witness with 0's so that its length is exactly $2^{|x|^c}$).

Define the **padded language**

$$L_{\text{pad}} = \{ x \| 1^{2^{|x|^c}} : x \in L \}.$$

Claim 12. $L_{\text{pad}} \in \text{NP}$.

Proof of Claim 12. On input $y = x \| 1^m$ for some prefix x . Let $n = |x|$ and $m = 2^{|x|^c}$. Let $N = |y| = |x| + m$. Let the witness $w_{\text{pad}} = \langle n, w \rangle$, where w is a witness for x , i.e. $V(x, w) = 1$. Since $|w| = O(N)$, we have $|w_{\text{pad}}| = O(\log N) + O(N) = O(N)$. Define a TM $V'(y, w_{\text{pad}})$ that accepts iff $V(x, w) = 1$. V runs in time polynomial in $|x| + |w| = O(N)$, the whole check runs in $\text{poly}(N)$ time using a witness of length $\text{poly}(N)$. Hence $L_{\text{pad}} \in \text{NP}$. $\square$

Claim 13. If $L_{\text{pad}} \in \text{P}$, then $L \in \text{EXP}$.

Proof of Claim 13. On input $y = x \| 1^m$ for some prefix x and $m = 2^{|x|^c}$. Let $N = |y| = |x| + m$ and A decide L_{pad} in time $\text{poly}(N)$. Since $|y| = O(2^{|x|^c})$, this whole procedure takes time $\text{poly}(2^{|x|^c}) = 2^{O(|x|^c)} \leq 2^{|x|^{c'}}$ for a suitable constant c' . Moreover $A(y) = 1 \iff y \in L_{\text{pad}} \iff x \in L$. So this decides L in time $2^{|x|^{c'}}$, i.e. $L \in \text{EXP}$. $\square$

Combining the two claims: if $\text{P} = \text{NP}$, then $L_{\text{pad}} \in \text{NP} = \text{P}$, and so by the second claim $L \in \text{EXP}$. As L was an arbitrary language in NEXP , this shows $\text{NEXP} \subseteq \text{EXP}$, hence $\text{EXP} = \text{NEXP}$. $\square$

References

[AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.